

Data and Artificial Intelligence

Cyber Shujaa Program

Week 1 Assignment

Web Scraping and Data Handling in Python

Student Name: Hope Njoki Nyambura

Student ID: cs-eh03-25100

Table of Contents

1. Introduction	2
2. Objectives.....	2
3. Tasks Completed	3
Step 1: Import Libraries	3
Step 2: Set URL of Website	3
Step 3: Use BeautifulSoup to Extract HTML Content.....	4
Step 4: Extract the Table with the Hockey Scores	4
Step 5: Extract the Column Headings.....	5
Step 6: Save the Column Headings into a Pandas DataFrame.....	5
Step 7: Extract the Data Row by Row	6
Step 8: Show the First Few Rows of the DataFrame	6
Step 9: Save and Download the DataFrame as CSV.....	7
4. Conclusion.....	7
5. Link to Google Colab Notebook	7

1. Introduction

This assignment was aimed at building hands-on skills in web scraping using Python on Google Colab. It provided an opportunity to use Python libraries such as Requests, BeautifulSoup, and Pandas to gather, parse, clean, and store structured data. In addition, the task helped me understand the end-to-end workflow of automating data extraction from retrieving a live webpage, to transforming the HTML into structured tables, and finally exporting the results into a reusable format. The assignment also emphasized the importance of documenting the process with screenshots as evidence of work completed.

2. Objectives

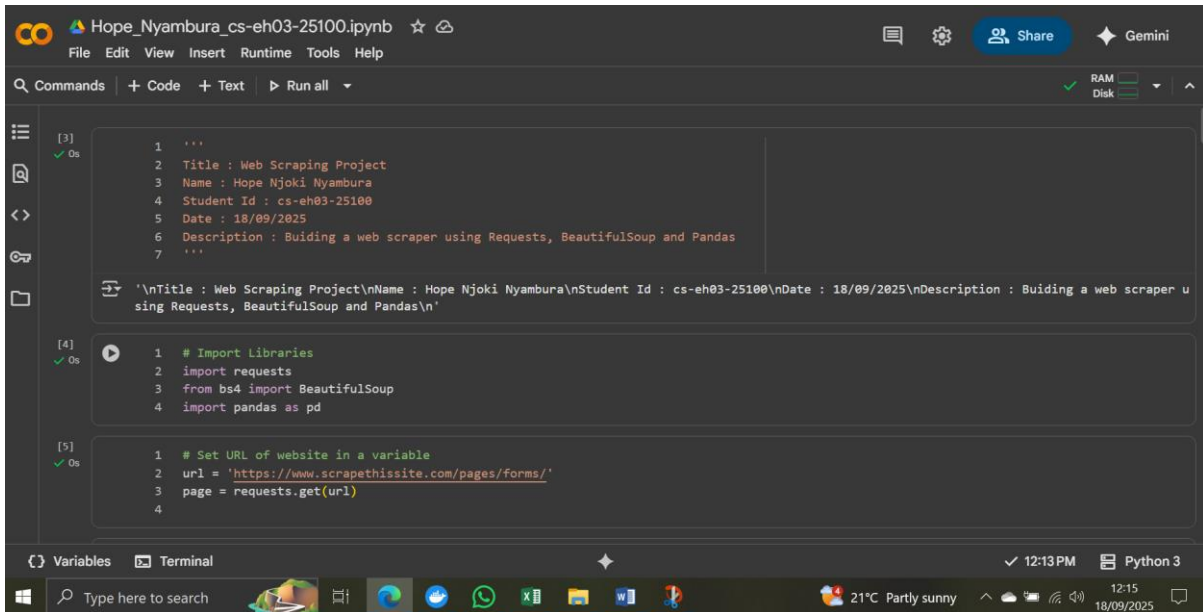
The main objectives of this assignment were:

- To gain practical experience with Python coding on Google Colab.
- To apply the Requests and BeautifulSoup libraries in order to extract data from a live web page.
- To parse and clean the extracted data so it can be analyzed effectively.
- To organize the structured data into a Pandas DataFrame for easier manipulation.
- To export the dataset to a .csv file, making it reusable for further analysis or reporting.

3. Tasks Completed

Step 1: Import Libraries

The first step was to import the libraries required for web scraping and data handling. I used Requests for sending HTTP requests, BeautifulSoup for parsing and extracting data from the HTML content, and Pandas for organizing the extracted data into a DataFrame.



```

[3] ✓ 0s
1 '''
2 Title : Web Scraping Project
3 Name : Hope Njoki Nyambura
4 Student Id : cs-eh03-25100
5 Date : 18/09/2025
6 Description : Building a web scraper using Requests, BeautifulSoup and Pandas
7 '''

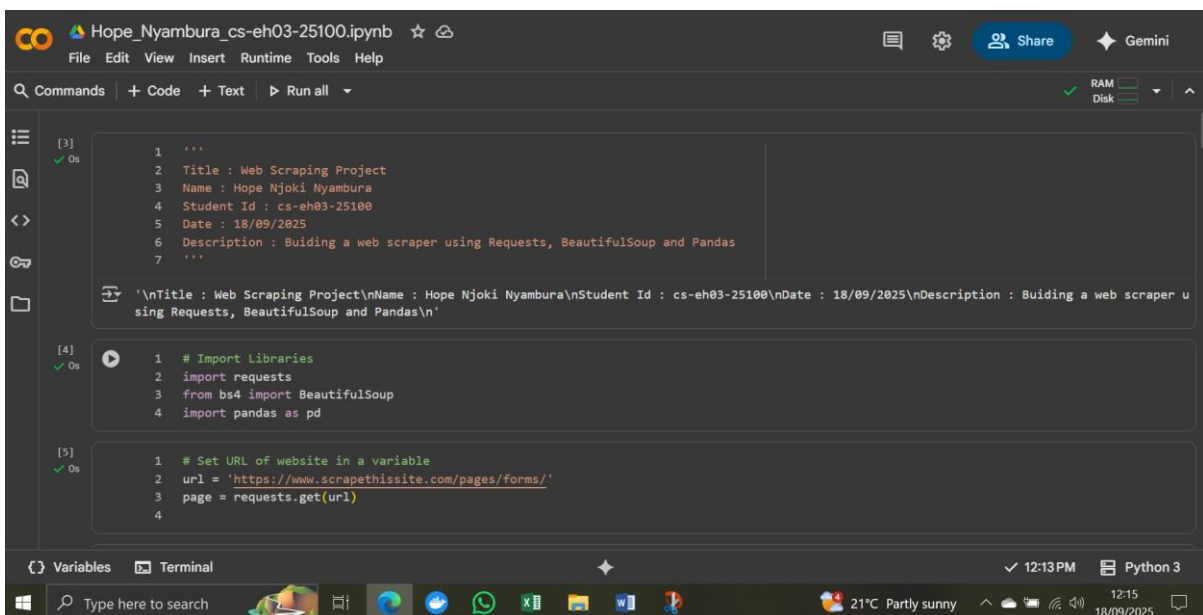
'\nTitle : Web Scraping Project\nName : Hope Njoki Nyambura\nStudent Id : cs-eh03-25100\nDate : 18/09/2025\nDescription : Building a web scraper u
sing Requests, BeautifulSoup and Pandas\n'

[4] ✓ 0s
1 # Import Libraries
2 import requests
3 from bs4 import BeautifulSoup
4 import pandas as pd

[5] ✓ 0s
1 # Set URL of website in a variable
2 url = 'https://www.scrapethissite.com/pages/forms/'
3 page = requests.get(url)
4
  
```

Step 2: Set URL of Website

In this step, I defined the target URL of the webpage to be scraped and stored it in a variable. I then used the requests.get() method to send an HTTP request to the server and retrieve the webpage content for further processing.



```

[3] ✓ 0s
1 '''
2 Title : Web Scraping Project
3 Name : Hope Njoki Nyambura
4 Student Id : cs-eh03-25100
5 Date : 18/09/2025
6 Description : Building a web scraper using Requests, BeautifulSoup and Pandas
7 '''

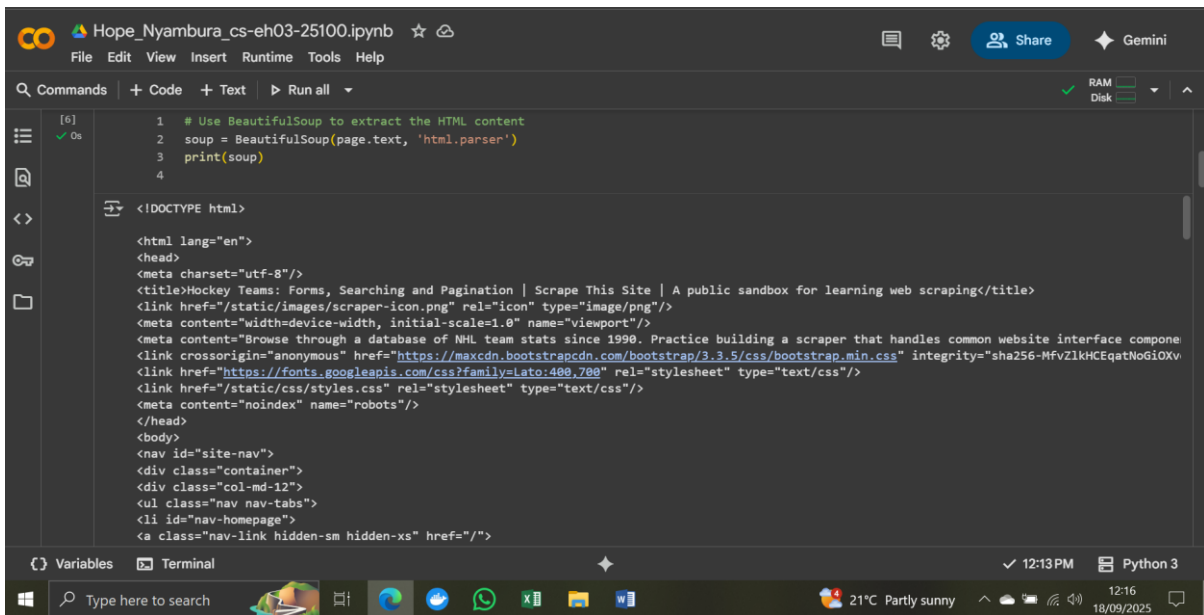
'\nTitle : Web Scraping Project\nName : Hope Njoki Nyambura\nStudent Id : cs-eh03-25100\nDate : 18/09/2025\nDescription : Building a web scraper u
sing Requests, BeautifulSoup and Pandas\n'

[4] ✓ 0s
1 # Import Libraries
2 import requests
3 from bs4 import BeautifulSoup
4 import pandas as pd

[5] ✓ 0s
1 # Set URL of website in a variable
2 url = 'https://www.scrapethissite.com/pages/forms/'
3 page = requests.get(url)
4
  
```

Step 3: Use BeautifulSoup to Extract HTML Content

After retrieving the webpage with `requests.get()`, I parsed the raw HTML content using BeautifulSoup with the `html.parser` argument. This allowed me to navigate the structure of the webpage and prepare the data for extraction. Printing the parsed output confirmed that the HTML was successfully loaded into BeautifulSoup.



```

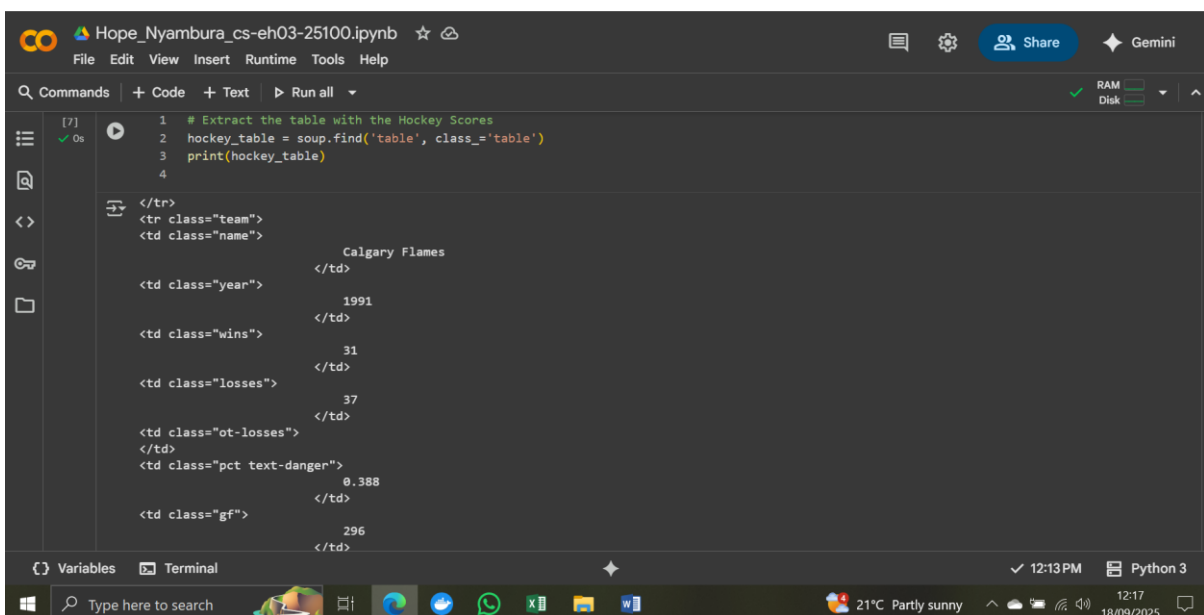
1 # Use BeautifulSoup to extract the HTML content
2 soup = BeautifulSoup(page.text, 'html.parser')
3 print(soup)
4
<!DOCTYPE html>

<html lang="en">
<head>
<meta charset="utf-8"/>
<title>Hockey Teams: Forms, Searching and Pagination | Scrape This Site | A public sandbox for learning web scraping</title>
<link href="/static/images/scraper-icon.png" rel="icon" type="image/png"/>
<meta content="width=device-width, initial-scale=1.0" name="viewport"/>
<meta content="Browse through a database of NHL team stats since 1990. Practice building a scraper that handles common website interface compone">
<link crossorigin="anonymous" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.5/css/bootstrap.min.css" integrity="sha256-MfvZlkHCEqatNoGiOxV">
<link href="https://fonts.googleapis.com/css?family=Lato:400,700" rel="stylesheet" type="text/css"/>
<link href="/static/css/styles.css" rel="stylesheet" type="text/css"/>
<meta content="noindex" name="robots"/>
</head>
<body>
<nav id="site-nav">
<div class="container">
<div class="col-md-12">
<ul class="nav nav-tabs">
<li id="nav-homepage">
<a class="nav-link hidden-sm hidden-xs" href="/">

```

Step 4: Extract the Table with the Hockey Scores

Using the parsed HTML, I located the hockey scores table by searching for the `<table>` element with the class attribute `"table"`. This step ensured that I isolated the section of the webpage containing the data of interest, which would later be used for extracting headers and row values.



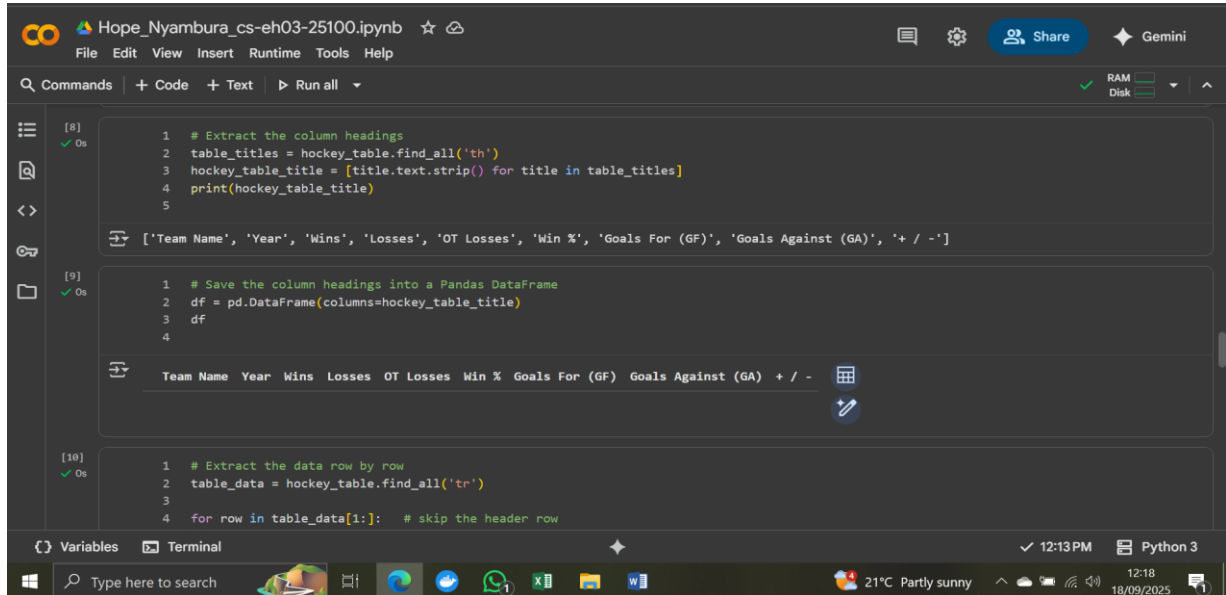
```

1 # Extract the table with the Hockey Scores
2 hockey_table = soup.find('table', class_='table')
3 print(hockey_table)
4
</tr>
<tr class="team">
<td class="name">
    Calgary Flames
</td>
<td class="year">
    1991
</td>
<td class="wins">
    31
</td>
<td class="losses">
    37
</td>
<td class="ot-losses">
</td>
<td class="pct text-danger">
    0.388
</td>
<td class="gf">
    296
</td>

```

Step 5: Extract the Column Headings

Once the table was located, I extracted all the column headers by selecting the <th> elements within the table. These headers were then stripped of any extra whitespace and stored in a Python list, which provided the structure for building the DataFrame.



```
[8]
1 # Extract the column headings
2 table_titles = hockey_table.find_all('th')
3 hockey_table_title = [title.text.strip() for title in table_titles]
4 print(hockey_table_title)
5

['Team Name', 'Year', 'Wins', 'Losses', 'OT Losses', 'Win %', 'Goals For (GF)', 'Goals Against (GA)', '+ / -']

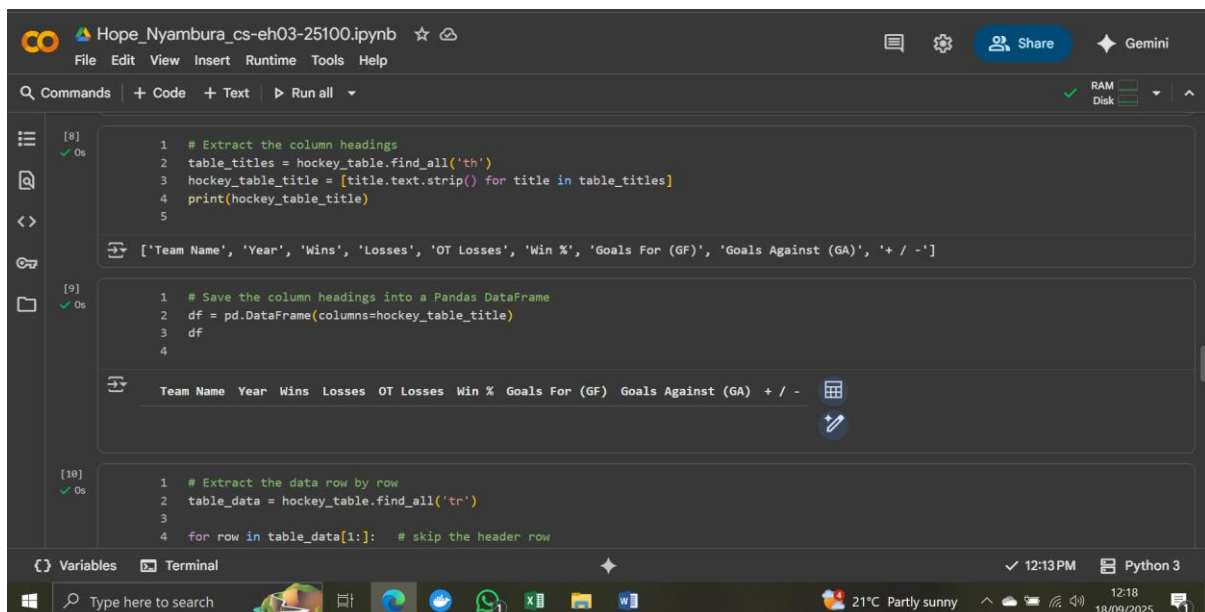
[9]
1 # Save the column headings into a Pandas DataFrame
2 df = pd.DataFrame(columns=hockey_table_title)
3 df
4

Team Name Year Wins Losses OT Losses Win % Goals For (GF) Goals Against (GA) + / -

[10]
1 # Extract the data row by row
2 table_data = hockey_table.find_all('tr')
3
4 for row in table_data[1:]: # skip the header row
```

Step 6: Save the Column Headings into a Pandas DataFrame

Using the extracted column headers, I created an empty Pandas DataFrame. This provided a structured container with labeled columns, ready to store the row data that would be scraped from the table.



```
[8]
1 # Extract the column headings
2 table_titles = hockey_table.find_all('th')
3 hockey_table_title = [title.text.strip() for title in table_titles]
4 print(hockey_table_title)
5

['Team Name', 'Year', 'Wins', 'Losses', 'OT Losses', 'Win %', 'Goals For (GF)', 'Goals Against (GA)', '+ / -']

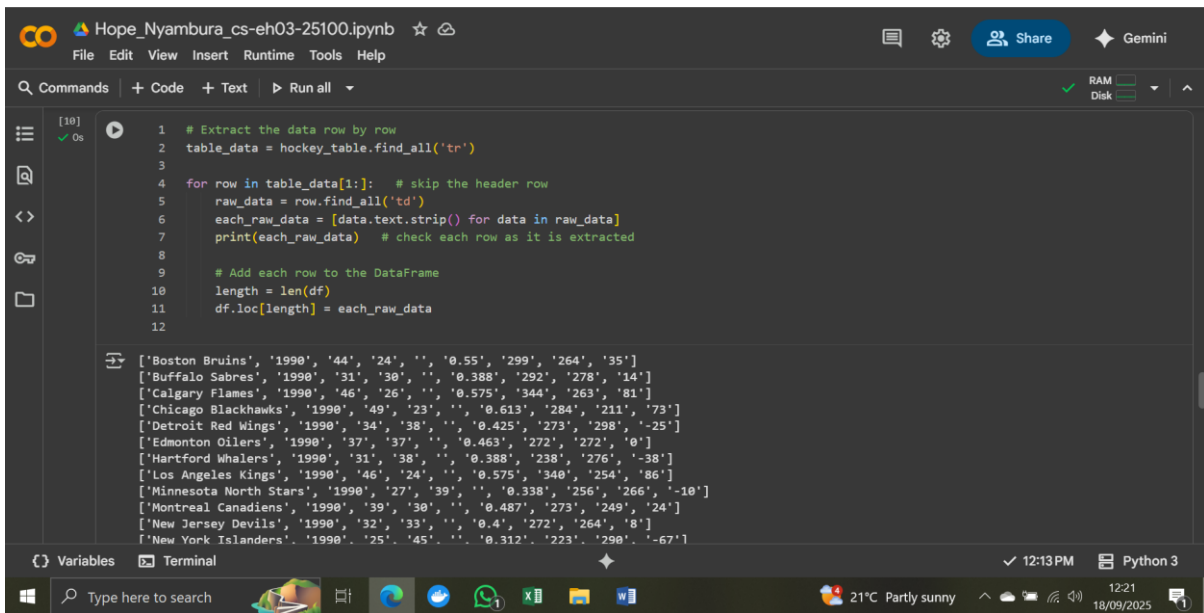
[9]
1 # Save the column headings into a Pandas DataFrame
2 df = pd.DataFrame(columns=hockey_table_title)
3 df
4

Team Name Year Wins Losses OT Losses Win % Goals For (GF) Goals Against (GA) + / -

[10]
1 # Extract the data row by row
2 table_data = hockey_table.find_all('tr')
3
4 for row in table_data[1:]: # skip the header row
```

Step 7: Extract the Data Row by Row

I iterated through each row (<tr>) of the table, skipping the header row, and extracted the cell values (<td>). Each value was stripped of unnecessary whitespace and stored as a list. These lists were then appended one by one into the Pandas DataFrame, gradually populating it with the hockey scores data.



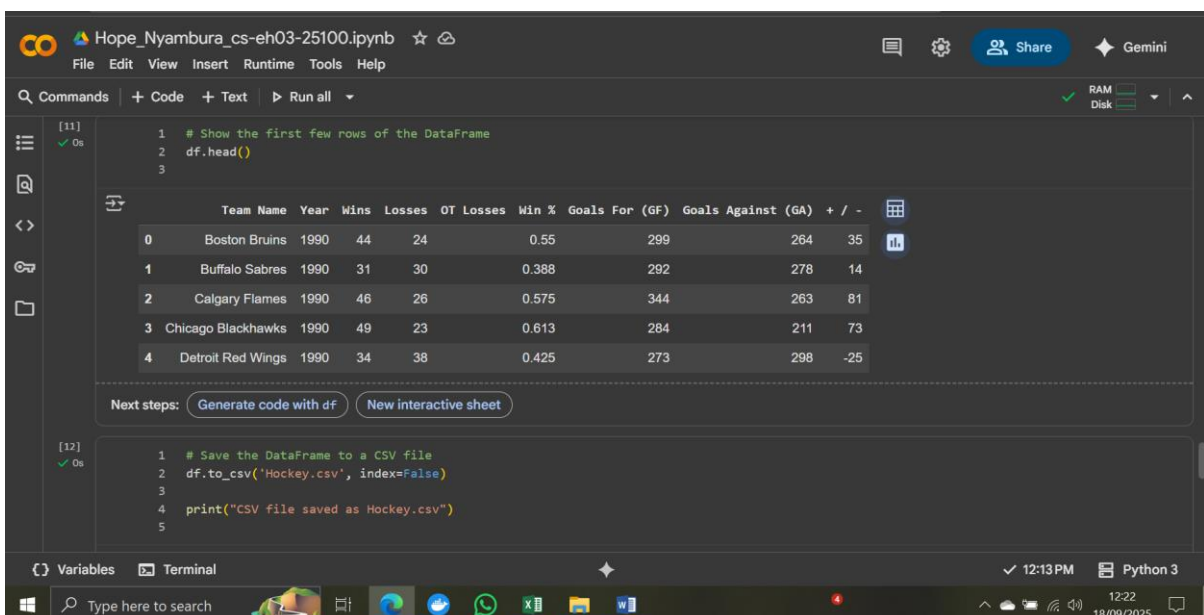
```

1 # Extract the data row by row
2 table_data = hockey_table.find_all('tr')
3
4 for row in table_data[1:]: # skip the header row
5     raw_data = row.find_all('td')
6     each_raw_data = [data.text.strip() for data in raw_data]
7     print(each_raw_data) # check each row as it is extracted
8
9 # Add each row to the DataFrame
10 length = len(df)
11 df.loc[length] = each_raw_data
12
13 ['Boston Bruins', '1990', '44', '24', '', '0.55', '299', '264', '35']
14 ['Buffalo Sabres', '1990', '31', '30', '', '0.388', '292', '278', '14']
15 ['Calgary Flames', '1990', '46', '26', '', '0.575', '344', '263', '81']
16 ['Chicago Blackhawks', '1990', '49', '23', '', '0.613', '284', '211', '73']
17 ['Detroit Red Wings', '1990', '34', '38', '', '0.425', '273', '298', '-25']
18 ['Edmonton Oilers', '1990', '37', '37', '', '0.463', '272', '272', '0']
19 ['Hartford Whalers', '1990', '31', '38', '', '0.388', '238', '276', '-38']
20 ['Los Angeles Kings', '1990', '46', '24', '', '0.575', '340', '254', '86']
21 ['Minnesota North Stars', '1990', '27', '39', '', '0.338', '256', '266', '-10']
22 ['Montreal Canadiens', '1990', '39', '30', '', '0.487', '273', '249', '24']
23 ['New Jersey Devils', '1990', '32', '33', '', '0.4', '272', '264', '8']
24 ['New York Islanders', '1990', '25', '45', '', '0.312', '223', '290', '-67']

```

Step 8: Show the First Few Rows of the DataFrame

After populating the DataFrame with the scraped data, I displayed the first few rows using the .head() method. This step allowed me to quickly inspect the structure and content of the DataFrame, verify that the data had been correctly extracted, and ensure the column headers aligned properly with the row values.



```

1 # Show the first few rows of the DataFrame
2 df.head()
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```

	Team Name	Year	Wins	Losses	OT Losses	Win %	Goals For (GF)	Goals Against (GA)	+ / -
0	Boston Bruins	1990	44	24		0.55	299	264	35
1	Buffalo Sabres	1990	31	30		0.388	292	278	14
2	Calgary Flames	1990	46	26		0.575	344	263	81
3	Chicago Blackhawks	1990	49	23		0.613	284	211	73
4	Detroit Red Wings	1990	34	38		0.425	273	298	-25

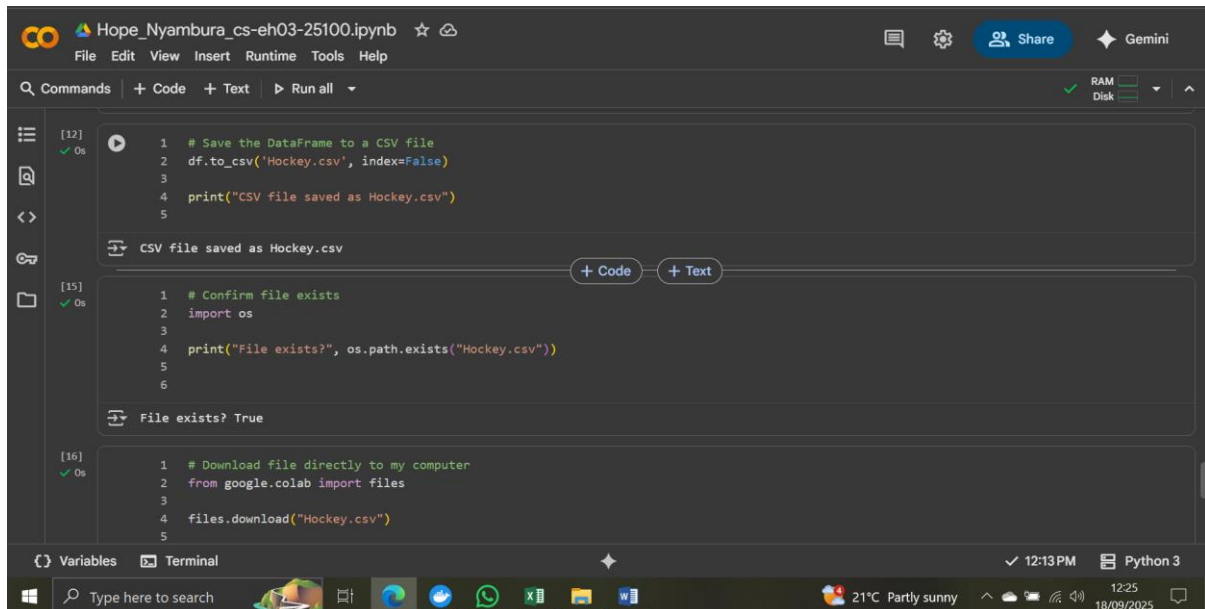
```

1 # Save the DataFrame to a CSV file
2 df.to_csv('Hockey.csv', index=False)
3
4 print("CSV file saved as Hockey.csv")
5

```

Step 9: Save and Download the DataFrame as CSV

After verifying that the DataFrame contained all the scraped data correctly, I exported it to a CSV file named Hockey.csv using Pandas' to_csv() method. This step ensured the data was stored in a reusable format. I then confirmed that the file had been created successfully and downloaded it directly to my computer for submission.



```

[12] 1 # Save the DataFrame to a CSV file
      2 df.to_csv('Hockey.csv', index=False)
      3
      4 print("CSV file saved as Hockey.csv")
      5

CSV file saved as Hockey.csv

[15] 1 # Confirm file exists
      2 import os
      3
      4 print("File exists?", os.path.exists("Hockey.csv"))
      5
      6

File exists? True

[16] 1 # Download file directly to my computer
      2 from google.colab import files
      3
      4 files.download("Hockey.csv")
      5
  
```

4. Conclusion

This assignment provided a valuable introduction to web scraping and the end-to-end process of data extraction. I gained practical experience in retrieving structured data from websites, parsing HTML content using BeautifulSoup, and organizing it efficiently with Pandas. Exporting the data to a CSV file highlighted how web-scraped information can be stored and reused for further analysis or reporting. Overall, the exercise strengthened my understanding of automated data collection workflows and reinforced the importance of careful documentation and verification throughout the process.

5. Link to Google Colab Notebook

[<https://colab.research.google.com/drive/1fXNAbQ2FdOBNoYhJ2bmgZO1ZllqPy0m9?usp=sharing>]